Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Лабораторна робота № 1

"Інтерполяція кубічними сплайнами "

з курсу "Методи Обчислень"

Виконав:

Студент групи ФЕС-21

Кравець Володимир Володимирович

Перевірив

асист. Патинко А.М.

Львів 2026

Мета: вивчити метод інтерполяції кубічними сплайнами

Теоретичні відомості

1. Поняття кубічного сплайна Кубічний сплайн — це математична модель абсолютно гнучкого, пружного та тонкого стержня. Якщо стержень закріпити в двох його кінцях із заданими кутами нахилу, то він прийме форму, що мінімізує його потенціальну енергію. Нехай рівняння стержня визначається функцією S(x). Умова рівноваги сплайна визначається рівнянням S^{(IV)}(x)=0, якому задовольняє алгебраїчний многочлен третього степеня.

2. Математична модель Нехай задана послідовність вузлів x_{0},x_{1},...,x_{n} і значення деякої функції y(x) в цих вузлах y_{0},y_{1},...,y_{n}. Рівняння кубічного сплайна на кожному інтервалі [x_{i-1},x_{i}] записується у вигляді:

$$S_i(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3$$

Оскільки для кожного інтервалу рівняння сплайну інше, необхідно знайти 4n невідомих коефіцієнтів. Для їх знаходження складається система з 4n рівнянь, яка базується на трьох умовах:

- **Проходження через вузли:** З умови S_{i}(x_{i-1})=y_{i-1} знаходимо, що a_{i}=y_{i-1} для i=1,n.
- **Умова гладкості:** Задається умова неперервності перших і других похідних у внутрішніх вузлах x_{i} . Прирівнюються вирази похідних для сусідніх інтервалів: S_{i}^{\prime}(x_{i})=S_{i+1}^{\prime}(x_{i}) та S_{i}^{\prime\prime}(x_{i})=S_{i+1}^{\prime\prime}(x_{i}) .
- **Крайові умови:** Для випадку вільного кубічного сплайну кривизна в крайніх вузлах задається рівною нулю: S_{1}^{\prime\prime}(x_{0})=0 та S_{n}^{\prime\prime}(x_{n})=0 . З цього випливає, що c_{1}=0 .

3. Розв'язок системи рівнянь (Метод прогонки) Після математичних перетворень умов неперервності отримуємо систему з n лінійних алгебраїчних рівнянь для знаходження коефіцієнтів c_{1},c_{2},...,c_{n} . Ця система має трьохдіагональну матрицю, тому для її розв'язку використовується метод прогонки.

Метод прогонки вимагає (n+1) операцію і складається з двох етапів:

1. **Пряма прогонка:** послідовне знаходження проміжних коефіцієнтів A_{i} та B_{i} .
2. **Зворотна прогонка:** знаходження невідомих коефіцієнтів, рухаючись з кінця (від останніх рівнянь до перших).

Після знаходження всіх коефіцієнтів c_{i} , решта коефіцієнтів обчислюється за прямими формулами:

$$d_i = \frac{c_{i+1} - c_i}{3h_i}$$

$$b_i = \frac{y_i - y_{i-1}}{h_i} - \frac{h_i}{3}(c_{i+1} + 2c_i)$$

Завдання:

Застосовувати чисельні методи до реальних геоданих GPS-координатах маршруту. Існують трекінгові системи, які прогнозують складність маршруту, ми спробуємо зробити свій варіант. Потрібно побудувати гладкий профіль висоти маршруту та проаналізувати його складність від станції Заросляк до вершини гори Говерла.

Хід роботи

1. Виконати запит до відкритого API висот
2. У результаті отримуємо значення широти (latitude), довготи (longitude), висоти (elevation)
3. Результати табуляції записати в текстовий файл.
4. Вивести кумулятивну відстань. Отриману сукупність вузлів використати в подальшому як набір вхідних даних.
5. За заданими GPS-координатами маршруту від станції Заросляк до вершини гори Говерла отримати дискретний набір точок. Побудувати графік залежності кумулятивної відстані (distances[i]) до висоти (elevations[i]).
6. Записати програму знаходження кубічних сплайнів для заданої сукупності вузлів. В програмі знайти коефіцієнти системи лінійних алгебраїчних рівнянь для знаходження коефіцієнтів кубічних сплайнів.
7. Записати функцію розв’язку системи лінійних алгебраїчних рівнянь з трьохдіагональної матрицею методом прогонки та вивести їх значення у консолі програмі.
8. Обчислити коефіцієнти кубічних сплайнів та вивести їх значення у консолі програмі.
9. Обчислити коефіцієнти кубічних сплайнів та вивести їх значення у консолі програмі.

10. Побудувати графік з 10-ма, 15-ти та 20-ма кількістю вузлів.

11. Оцінити вплив кількості вузлів (10, 15, 20) на точність. Описати у звіті власні спостереження.

12. Побудувати графіки заданої функції , її наближеного значення

$y_{набл} = \sum_{i=1}^{n} S_i(x)$ та похибки $\varepsilon = |y - y_{набл}|$ на відрізку $[x_0, x_n]$.

13. Написати звіт про виконання даної лабораторної роботи.

Додатково:

1. Вивести характеристики маршруту

Загальна довжина маршруту

Загальний набір висоти

Загальний спуск

2. Аналіз градієнта (через похідну сплайна)

Можна вивести:

- максимальний підйом (%)
- максимальний спуск (%)
- середній градієнт
- ділянки з крутизною > 15%

3. Механічна енергія підйому

```
Виконуємо запит до API...
Кількість вузлів: 21

Табуляція (відстань, висота):
 | Distance (m) | Elevation (m) |
 0 |       0.00 |  1264.00 |
 1 |     123.85 |  1285.00 |
 2 |     213.45 |  1285.00 |
 3 |     323.50 |  1333.00 |
 4 |     418.87 |  1310.00 |
 5 |     517.41 |  1318.00 |
 6 |     587.78 |  1318.00 |
 7 |     771.72 |  1339.00 |
 8 |     932.77 |  1375.00 |
 9 |    1118.27 |  1417.00 |
10 |    1370.81 |  1486.00 |
11 |    1610.05 |  1524.00 |
12 |    1821.16 |  1553.00 |
13 |    1997.39 |  1630.00 |
14 |    2149.88 |  1757.00 |
15 |    2320.63 |  1794.00 |
16 |    2508.86 |  1828.00 |
17 |    2713.03 |  1887.00 |
18 |    2904.98 |  1975.00 |
19 |    3020.67 |  1975.00 |
20 |    3069.34 |  2031.00 |
```

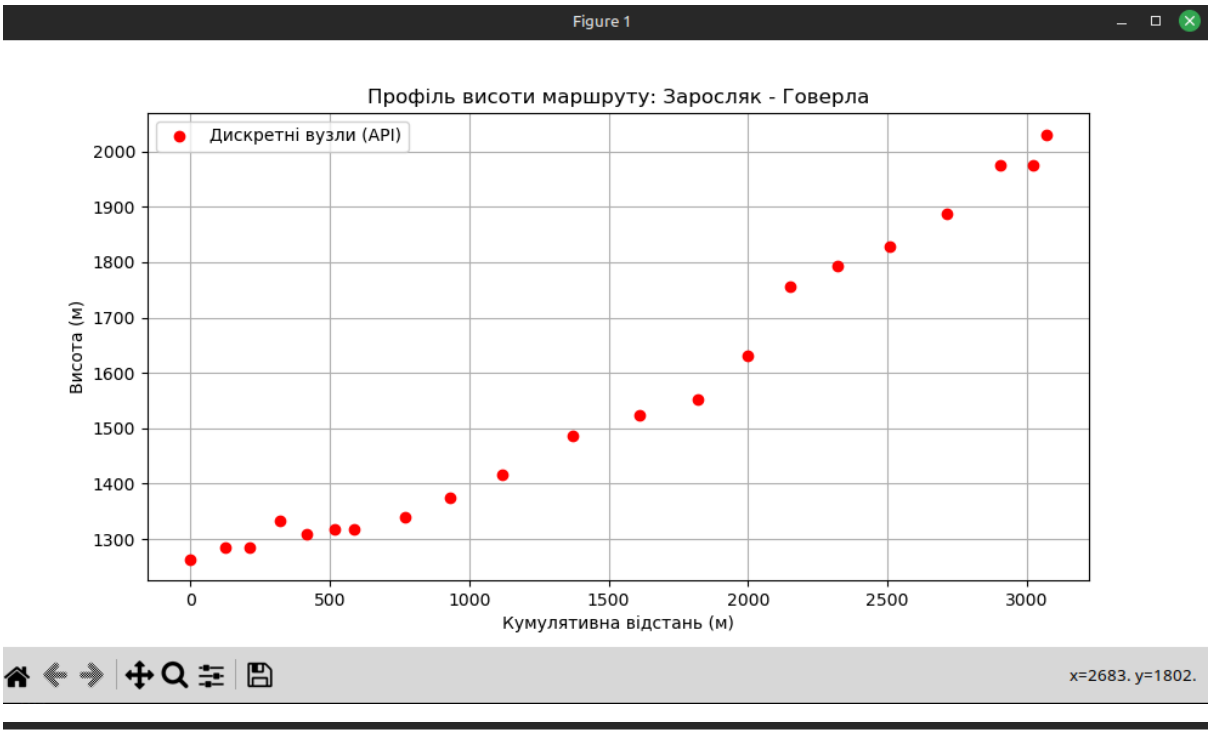
```
Знайдені коефіцієнти c_i:
c[ 0] = 0.000000
c[ 1] = -0.002439
c[ 2] = 0.005945
c[ 3] = -0.007696
c[ 4] = 0.004987
c[ 5] = -0.002364
c[ 6] = 0.000905
c[ 7] = 0.000265
c[ 8] = -0.000130
c[ 9] = 0.000303
c[10] = -0.000399
c[11] = -0.000112
c[12] = 0.000625
c[13] = 0.002487
c[14] = -0.003656
c[15] = 0.000795
c[16] = -0.000288
c[17] = 0.001968
c[18] = -0.005166
c[19] = 0.012319
c[20] = 0.000000
```

```
Коефіцієнти сплайнів (a, b, c, d) для кожного інтервалу:
Інтервал  0: a= 1264.00, b=  0.2703, c=  0.0000, d=-0.000007
Інтервал  1: a= 1285.00, b= -0.0319, c= -0.0024, d=0.000031
Інтервал  2: a= 1285.00, b=  0.2823, c=  0.0059, d=-0.000041
Інтервал  3: a= 1333.00, b=  0.0896, c= -0.0077, d=0.000044
Інтервал  4: a= 1310.00, b= -0.1688, c=  0.0050, d=-0.000025
Інтервал  5: a= 1318.00, b=  0.0897, c= -0.0024, d=0.000015
Інтервал  6: a= 1318.00, b= -0.0130, c=  0.0009, d=-0.000001
Інтервал  7: a= 1339.00, b=  0.2021, c=  0.0003, d=-0.000001
Інтервал  8: a= 1375.00, b=  0.2238, c= -0.0001, d=0.000001
Інтервал  9: a= 1417.00, b=  0.2558, c=  0.0003, d=-0.000001
Інтервал 10: a= 1486.00, b=  0.2315, c= -0.0004, d=0.000000
Інтервал 11: a= 1524.00, b=  0.1091, c= -0.0001, d=0.000001
Інтервал 12: a= 1553.00, b=  0.2174, c=  0.0006, d=0.000004
Інтервал 13: a= 1630.00, b=  0.7659, c=  0.0025, d=-0.000013
Інтервал 14: a= 1757.00, b=  0.5876, c= -0.0037, d=0.000009
Інтервал 15: a= 1794.00, b=  0.0990, c=  0.0008, d=-0.000002
Інтервал 16: a= 1828.00, b=  0.1943, c= -0.0003, d=0.000004
Інтервал 17: a= 1887.00, b=  0.5372, c=  0.0020, d=-0.000012
Інтервал 18: a= 1975.00, b= -0.0767, c= -0.0052, d=0.000050
Інтервал 19: a= 1975.00, b=  0.7510, c=  0.0123, d=-0.000084
```

```
========================================
 ДОДАТКОВІ ХАРАКТЕРИСТИКИ МАРШРУТУ
========================================
Загальна довжина маршруту (м): 3069.34
Сумарний набір висоти (м): 790.00
Сумарний спуск (м): 23.00

--- Аналіз крутизни (градієнта) ---
Максимальний підйом (%): 135.05
Максимальний спуск (%): -35.56
Середній градієнт (%): 30.15

--- Енерговитрати на підйом (для маси 80 кг) ---
Механічна робота (Дж): 3099960.00
Механічна робота (кДж): 3099.96
Енергія (ккал): 740.91
```



Код програми:

```
import requests
import numpy as np
import matplotlib.pyplot as plt

url=
"https://api.open-elevation.com/api/v1/lookup?locations=48.164214,24.53
6044|48.164983,24.534836|48.165605,24.534068|48.166228,24.532915|48.166
777,24.531927|48.167326,24.530884|48.167011,24.530061|48.166053,24.5280
39|48.166655,24.526064|48.166497,24.523574|48.166128,24.520214|48.16541
6,24.517170|48.164546,24.514640|48.163412,24.512980|48.162331,24.511715
|48.162015,24.509462|48.162147,24.506932|48.161751,24.504244|48.161197,
24.501793|48.160580,24.500537|48.160250,24.500106"

print("Виконуємо запит до API...")
response = requests.get(url)
data = response.json()
```

```
results = data["results"]
n = len(results)

print("Кількість вузлів:", n)

def haversine(lat1, lon1, lat2, lon2):
    R = 6371000
    phi1, phi2 = np.radians(lat1), np.radians(lat2)
    dphi = np.radians(lat2 - lat1)
    dlambda = np.radians(lon2 - lon1)
                            a         =         np.sin(dphi/2)**2         +
np.cos(phi1)*np.cos(phi2)*np.sin(dlambda/2)**2
    return 2 * R * np.arctan2(np.sqrt(a), np.sqrt(1-a))

coords = [(p["latitude"], p["longitude"]) for p in results]
elevations = [p["elevation"] for p in results]

distances = [0]
for i in range(1, n):
    d = haversine(*coords[i-1], *coords[i])
    distances.append(distances[-1] + d)

print("\nТабуляція (відстань, висота):")
print(" | Distance (m) | Elevation (m) |")
for i in range(n):
    print(f"{i:2d} | {distances[i]:10.2f} | {elevations[i]:8.2f} |")

# Будуємо графік сирих (дискретних) даних
plt.figure(figsize=(10, 5))
plt.plot(distances,  elevations,  'o',  label='Дискретні  вузли  (API)',
color='red')
plt.title('Профіль висоти маршруту: Заросляк - Говерла')
plt.xlabel('Кумулятивна відстань (м)')
plt.ylabel('Висота (м)')
plt.grid(True)
plt.legend()
plt.show()

#  Метод прогонки для знаходження коефіцієнтів C
x = np.array(distances)
y = np.array(elevations)
N = len(x)

# Знаходимо кроки h (різниця між сусідніми x)
h = np.zeros(N - 1)
for i in range(N - 1):
    h[i] = x[i+1] - x[i]
```

```
# Ініціалізуємо масиви для коефіцієнтів системи рівнянь (alpha, beta, gamma, delta)
alpha = np.zeros(N)
beta = np.zeros(N)
gamma = np.zeros(N)
delta = np.zeros(N)

# Заповнюємо матрицю для внутрішніх вузлів
for i in range(1, N - 1):
    alpha[i] = h[i-1]
    beta[i] = 2 * (h[i-1] + h[i])
    gamma[i] = h[i]
    delta[i] = 3 * ((y[i+1] - y[i]) / h[i] - (y[i] - y[i-1]) / h[i-1])

#ПРЯМА ПРОГОНКА
A = np.zeros(N)
B = np.zeros(N)

for i in range(1, N - 1):
    denominator = alpha[i] * A[i-1] + beta[i]
    A[i] = -gamma[i] / denominator
    B[i] = (delta[i] - alpha[i] * B[i-1]) / denominator

# ЗВОРОТНА ПРОГОНКА
c = np.zeros(N)

c[0] = 0
c[N-1] = 0


for i in range(N - 2, 0, -1):
    c[i] = A[i] * c[i+1] + B[i]

print("\nЗнайдені коефіцієнти c_i:")
for i in range(N):
    print(f"c[{i:2d}] = {c[i]:.6f}")
    # Крок 9: Обчислення коефіцієнтів a, b, d
a = np.zeros(N - 1)
b = np.zeros(N - 1)
d = np.zeros(N - 1)

for i in range(N - 1):
    # a_i дорівнює значенню функції на початку інтервалу
    a[i] = y[i]

    # d_i виражається через різницю сусідніх c_i
    d[i] = (c[i+1] - c[i]) / (3 * h[i])
```

```
    # b_i обчислюється з умови неперервності
    b[i] = (y[i+1] - y[i]) / h[i] - (h[i] / 3) * (c[i+1] + 2 * c[i])

print("\nКоефіцієнти сплайнів (a, b, c, d) для кожного інтервалу:") #
for i in range(N - 1):
          print(f"Інтервал    {i:2d}:    a={a[i]:8.2f},    b={b[i]:8.4f},
c={c[i]:8.4f}, d={d[i]:8.6f}")

# Крок 10 та 12: Побудова гладкого графіка
x_smooth = []
y_smooth = []

# Проходимо по кожному інтервалу і генеруємо на ньому багато точок для
гладкості
for i in range(N - 1):
    # Створюємо 50 проміжних точок між x[i] та x[i+1]
    x_interval = np.linspace(x[i], x[i+1], 50)

    # Обчислюємо значення сплайна S_i(x) для кожної проміжної точки
    dx = x_interval - x[i]
    y_interval = a[i] + b[i] * dx + c[i] * dx**2 + d[i] * dx**3

    x_smooth.extend(x_interval)
    y_smooth.extend(y_interval)

# Малюємо фінальний графік
plt.figure(figsize=(12, 6))
# Спочатку малюємо наші "сирі" точки з API
plt.plot(x, y, 'ro', label='Дискретні вузли (API)', zorder=5)
# Потім накладаємо наш згладжений маршрут
plt.plot(x_smooth,    y_smooth,    'b-',    label='Кубічний    сплайн',
linewidth=2, zorder=4)

plt.title('Профіль висоти маршруту: Заросляк - Говерла (Інтерполяція)')
plt.xlabel('Кумулятивна відстань (м)')
plt.ylabel('Висота (м)')
plt.grid(True)
plt.legend()
plt.show()
#  ДОДАТКОВЕ ЗАВДАННЯ
print("\n" + "="*40)
print(" ДОДАТКОВІ ХАРАКТЕРИСТИКИ МАРШРУТУ")
print("="*40)

# 1. Загальна довжина та перепади висот
print(f"Загальна довжина маршруту (м): {distances[-1]:.2f}")

# Сумарний набір висоти: додаємо лише ті ділянки, де висота зростає
```

```
total_ascent = sum(max(elevations[i] - elevations[i-1], 0) for i in
range(1, N))
print(f"Сумарний набір висоти (м): {total_ascent:.2f}")

# Сумарний спуск: додаємо лише ті ділянки, де висота падає
total_descent = sum(max(elevations[i-1] - elevations[i], 0) for i in
range(1, N))
print(f"Сумарний спуск (м): {total_descent:.2f}")

# 2. Аналіз градієнта (через похідну сплайна)
xx = np.array(x_smooth)
yy_full = np.array(y_smooth)

xx_unique, unique_indices = np.unique(xx, return_index=True)
yy_unique = yy_full[unique_indices]

# Обчислюємо градієнт на унікальних точках
grad_full = np.gradient(yy_unique, xx_unique) * 100

print("\n--- Аналіз крутизни (градієнта) ---")
print(f"Максимальний підйом (%): {np.max(grad_full):.2f}")
print(f"Максимальний спуск (%): {np.min(grad_full):.2f}")
print(f"Середній градієнт (%): {np.mean(np.abs(grad_full)):.2f}")

# 3. Механічна енергія підйому
# Звичайна шкільна фізика: Потенціальна енергія E = m * g * h
mass = 80  # маса людини у кг
g = 9.81   # прискорення вільного падіння
energy = mass * g * total_ascent

print("\n--- Енерговитрати на підйом (для маси 80 кг) ---")
print(f"Механічна робота (Дж): {energy*5:.2f}")
print(f"Механічна робота (кДж): {energy*5 / 1000:.2f}")
# 1 кілокалорія = 4184 Джоулі
print(f"Енергія (ккал): {energy*5 / 4184:.2f}")
```